

Project 3: The Data Warehouse

Provisioning a Private, Secure AWS RDS Database and Accessing it via an SSH Tunnel

Submitter Name:	Khadija Ashraf
Course / Module:	DecodeLabs Cloud Computing Internship
Date Submitted:	17 th Sep 2026
Cloud Provider Used:	AWS RDS (MySQL) + EC2 Bastion Host
Bastion Host IP:	13.63.238.236

1. Objective

An e-commerce company is struggling to manage customer data using local spreadsheets. As their user base grows, they need a robust, scalable, and secure cloud database. This project demonstrates provisioning a managed AWS RDS (MySQL) database, isolating it in a private subnet with no public internet access, creating a structured table with proper constraints, populating it with records, and verifying data persistence, all accessed securely through an SSH tunnel via an existing EC2 instance acting as a bastion host.

2. Tools & Services Used

- Cloud Platform: Amazon Web Services (AWS) – RDS
- Database Engine: MySQL (Free Tier eligible, db.t3.micro)
- Bastion Host: Existing EC2 instance (DecodeLabs-WebServer, Ubuntu 24.04)
- Access Method: SSH Tunnel (local port forwarding) – database is NOT publicly accessible
- SQL Client: MySQL Workbench (or terminal mysql client)
- Local Machine: Windows (PowerShell)

3. Step 1 — Provision the RDS Database

A MySQL RDS instance was created using the Free Tier eligible db.t3.micro class, 20 GB of General Purpose SSD storage, and critically Public Access set to No, placing it inside the same VPC as the existing EC2 instance.

Screenshot 1: RDS Instance Created

Aurora and RDS

Databases

Create database

Free plan has access to limited features and resources

The free plan limits the features and resources that are available for RDS and Aurora databases. Upgrade your account plan to remove all limitations. [Learn more](#)

Upgrade plan

Engine options

Engine type

Aurora (MySQL Compatible)



Aurora (PostgreSQL Compatible)



MySQL



PostgreSQL



MariaDB



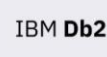
Oracle



Microsoft SQL Server



IBM Db2



Choose a database creation method

Full configuration

You set all of the configuration options, including ones for availability, security, backups, and maintenance.

Express configuration

Create and query an Aurora PostgreSQL serverless database with pre-configured settings to get started in seconds.

DB subnet group name

rds-ec2-db-subnet-group-1

New DB subnet group created.

Public access

Yes

RDS assigns a public IP address to the database. Amazon EC2 instances and other resources outside of the VPC can connect to your database. Resources inside the VPC can also one or more **VPC security groups** that specify which resources can connect to the database.

No

RDS doesn't assign a public IP address to the database. Only Amazon EC2 instances and other resources inside the VPC can connect to your database. Choose one or more **VPC security groups** that specify which resources can connect to the database.

VPC security group (firewall)

Choose one or more **VPC security groups** to allow access to your database. Make sure that the security group rules allow the appropriate incoming traffic.

Choose existing

Choose existing **VPC security groups**

Create new

Create new **VPC security group**

New VPC security group name

rds-private-sg

Amazon RDS will add a new **VPC security group** *rds-ec2-1* to allow connectivity with your compute resource.

Settings

Edition

☒ MySQL Community

Engine version [Info](#)

View the engine versions that support the following database features.

▼ Hide filters

☒ Show only versions that support the Multi-AZ DB cluster [Info](#)

Create a Multi-AZ DB cluster with one primary DB instance and two readable standby DB instances. Multi-AZ DB clusters provide up to 2x faster transaction commit latency and automatic failover in typically under 35 seconds.

☐ Show only versions that support the Amazon RDS Optimized Writes [Info](#)

Amazon RDS Optimized Writes improves write throughput by up to 2x at no additional cost.

Engine version

MySQL 8.4.9

☐ Enable RDS Extended Support [Info](#)

Amazon RDS Extended Support is a paid offering [↗](#). By selecting this option, you consent to being charged for this offering if you are running your database major version past the RDS end of standard support date for that version. Check the end of standard support date for your major version in the RDS for MySQL documentation [↗](#).

DB instance identifier [Info](#)

Type a name for your DB instance. The name must be unique across all DB instances owned by your AWS account in the current AWS Region.

decodelabs-db

The DB instance identifier is case-insensitive, but is stored as all lowercase (as in "mydbinstance"). Constraints: 1 to 63 alphanumeric characters or hyphens. First character must be a letter. Can't contain two consecutive hyphens. Can't end with a hyphen.

aws

Search

[Alt+S]

Europe (Stockholm)

Mariyam Ashraf (037169690641)

Mariyam%20Ashraf

Aurora and RDS

Databases

decodelabs-db

decodelabs-db

Modify

Actions

Summary

DB identifier

decodelabs-db

Status

Available

Role

Instance

Engine

MySQL Community

Recommendations

CPU

4.97%

Class

db.t4g.micro

Current activity

0 Connections

Region & AZ

eu-north-1b

Connectivity & security

Monitoring

Logs & events

Configuration

Zero-ETL integrations

Maintenance & backups

Data migrations

Tags

Recommendations

Connect using [Info](#)

☒ Code snippets

Use when connecting through SDK, APIs, or third-party tools including agents.

☐ CloudShell

Use for a quick access to AWS CLI that launches directly from the AWS Management Console.

☐ Endpoints

Use when connecting through any IDE interface.

Internet access gateway

Disabled

IAM Authentication

Disabled

Programming language

MySQL (macOS)

Endpoint type

Instance endpoint

Connection steps

CloudShell

Agent Toolkit for AWS

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

4. Step 2 — Lock Down Network Access

A dedicated security group was attached to the RDS instance, allowing inbound traffic on port 3306 (MySQL) ONLY from the security group of the EC2 bastion host, not from any public IP address. This ensures the database endpoint is unreachable directly from the internet.

Screenshot 2: RDS Security Group Inbound Rule

The screenshot displays the AWS Management Console interface for the RDS Security Group **sg-09283ee6854d226d5 - rds-ec2-1**. The console shows the following details:

- Security group name:** rds-ec2-1
- Security group ID:** sg-09283ee6854d226d5
- Description:** Security group attached to decode-labs-db to allow EC2 instances with specific security groups attached to connect to the database. Modification could lead to connection loss.
- VPC ID:** vpc-07a4d7d16d92483bf
- Owner:** 037169690641
- Inbound rules count:** 1 Permission entry
- Outbound rules count:** 0 Permission entries

The **Inbound rules** tab is selected, showing a single rule:

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
-	sg-00e17236bfa99c049	-	MySQL/Aurora	TCP	3306	sg-0918db07ae5ad618f...	Rule to

5. Step 3 — Open the SSH Tunnel

Since the RDS instance has no public address, a local SSH tunnel was opened through the EC2 bastion host to forward a local port to the RDS endpoint's port 3306.

```
PS D:\> ssh -i "decodeLabs-key.pem" -L 3306:&lt;RDS-ENDPOINT&gt;;3306  
ubuntu@13.63.238.236
```

Example:

```
PS D:\> ssh -i "decodeLabs-key.pem" -L 3306:decode-labs-db.abcd1234.eu-north-  
1.rds.amazonaws.com:3306 ubuntu@13.63.238.236
```

Screenshot 3: SSH Tunnel Command Running

```
D:\>ssh -i "decodeLabs-key.pem" -L 3306:decodeLabs-db.chy6oyskasy3.eu-north-1.rds.amazonaws.com:3306 ubuntu@13.63.238.23
6
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1017-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu Sep 17 11:08:59 UTC 2026

System load:  0.08               Temperature:   -273.1 C
Usage of /:   40.0% of 6.71GB    Processes:    118
Memory usage: 33%               Users logged in: 0
Swap usage:  0%                 IPv4 address for ens5: 172.31.35.187

 * Ubuntu Pro delivers the most comprehensive open source security and
   compliance features.

   https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

6 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

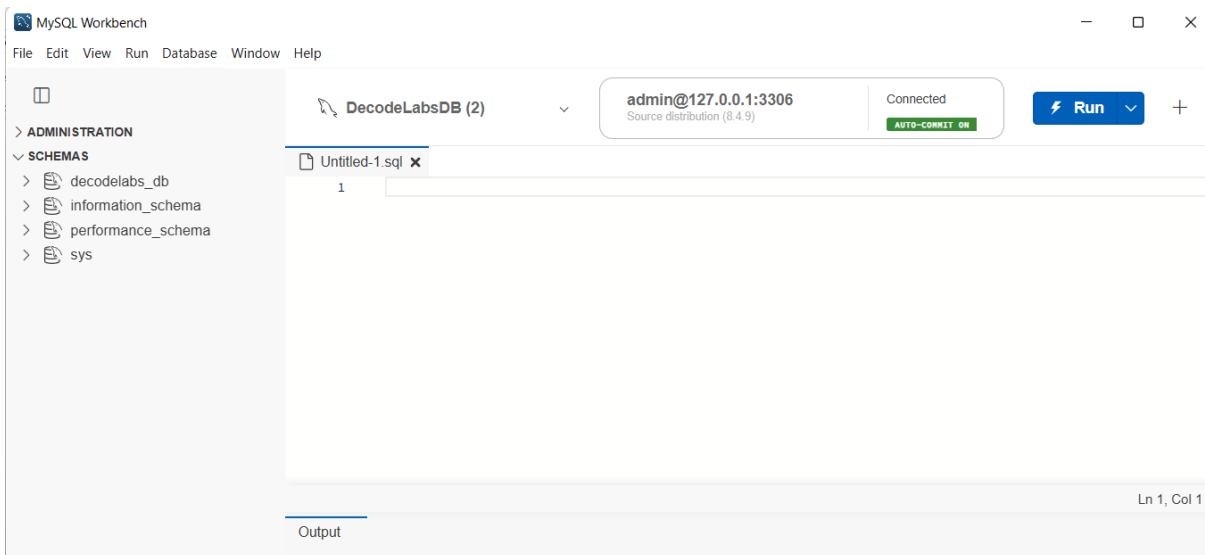
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

*** System restart required ***
Last login: Tue Sep 15 10:50:51 2026 from 119.73.102.23
ubuntu@ip-172-31-35-187:~$
```

6. Step 4 — Connect Using a SQL Client

With the tunnel open, a SQL client (MySQL Workbench) was configured to connect to 127.0.0.1:3306 (localhost) rather than the RDS endpoint directly, the tunnel transparently forwards this connection through to the real database.

Screenshot 4: SQL Client Connected

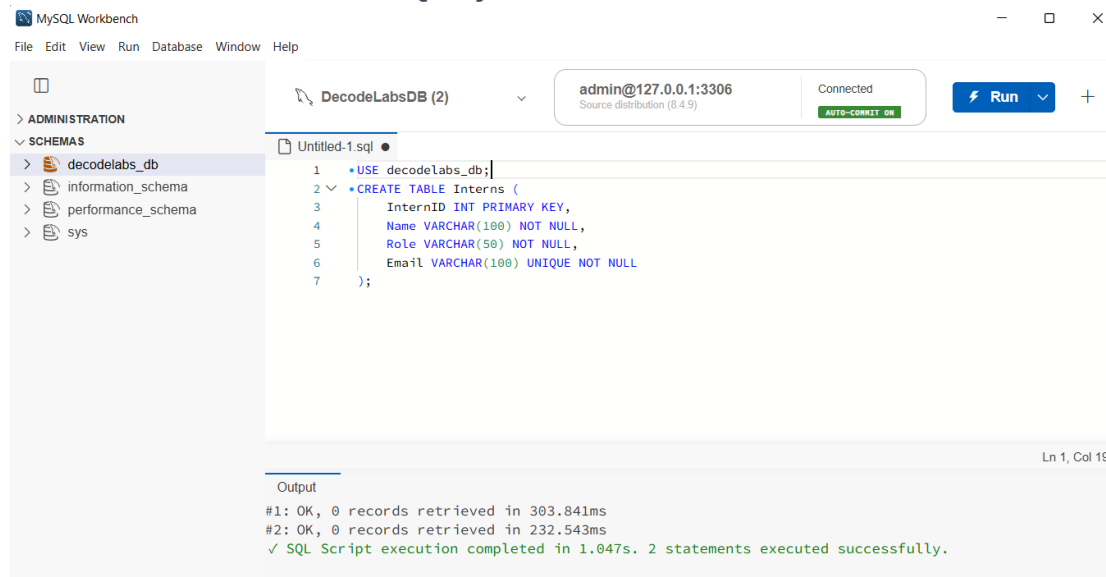


7. Step 5 — Create the Interns Table

A table named Interns was created with a primary key, NOT NULL constraints, and a UNIQUE constraint on the email field to enforce data integrity.

```
CREATE TABLE Interns (  
  InternID INT PRIMARY KEY,  
  Name VARCHAR(100) NOT NULL,  
  Role VARCHAR(50) NOT NULL, Email VARCHAR(100) UNIQUE NOT NULL  
);
```

Screenshot 5: Table Creation Query Executed

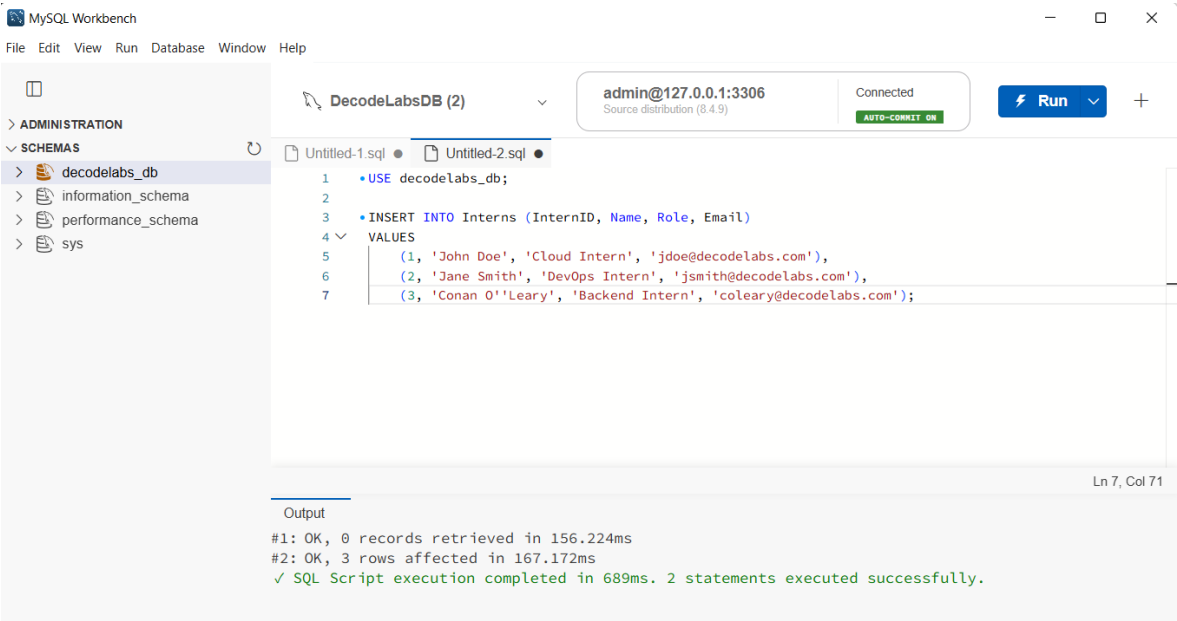


8. Step 6 — Insert Dummy Records

Multiple dummy intern records were inserted in a single batch statement to test data persistence.

```
INSERT INTO Interns (InternID, Name, Role, Email) VALUES  
(1, 'John Doe', 'Cloud Intern', 'jdoe@deodelabs.com'),  
(2, 'Jane Smith', 'DevOps Intern', 'jsmith@deodelabs.com'),  
(3, 'Conan O'Leary', 'Backend Intern', 'coleary@deodelabs.com');
```

Screenshot 6: Records Inserted

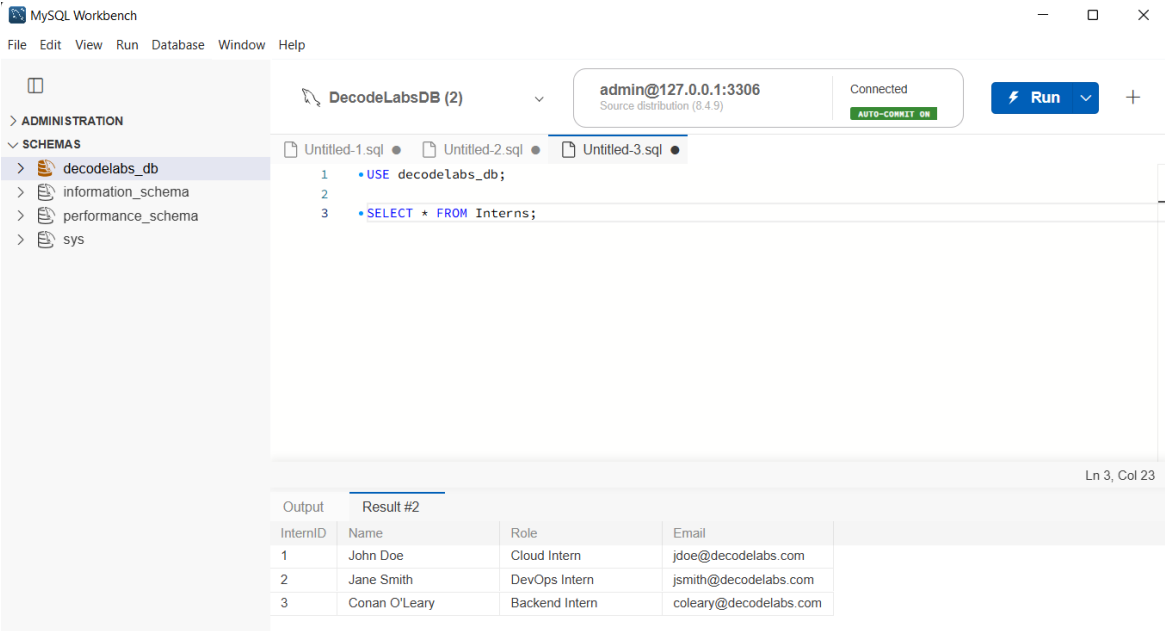


9. Step 7 — Verify Data Persistence

A SELECT query was run to confirm the inserted records are permanently stored and retrievable, proving the RDS instance, the tunnel, the schema, and the inserted data are all functioning correctly.

```
SELECT * FROM Interns;
```

Screenshot 7: SELECT Query Results



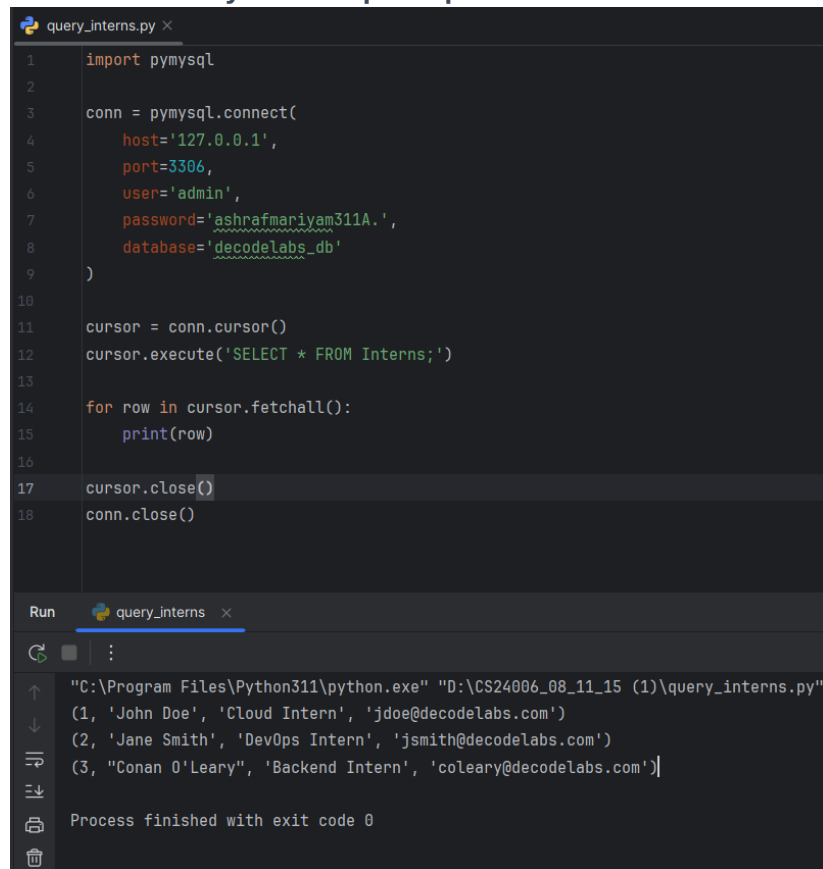
10. Step 8 — Bonus: Python Verification

As a bonus, the same verification was optionally repeated using a Python script connecting through the same local SSH tunnel via the PyMySQL library.

```
import pymysql

conn = pymysql.connect( host='127.0.0.1', port=3306, user='admin',
password='*****', database='decode labs_db' )
cursor = conn.cursor()
cursor.execute('SELECT * FROM
Interns;') for row in
cursor.fetchall(): print(row)
```

Screenshot 8 : Python Script Output

The screenshot shows a code editor with a file named 'query_interns.py' and a terminal window below it. The code in the editor connects to a MySQL database using pymysql, executes a query to select all data from the 'Interns' table, and prints the results. The terminal output shows the execution of the script, the command used, and the resulting data from the database query. The data is presented as a list of tuples, each containing an ID, a name, an email, and a role. The process finished with an exit code of 0.

```
1 import pymysql
2
3 conn = pymysql.connect(
4     host='127.0.0.1',
5     port=3306,
6     user='admin',
7     password='ashrafmariyam311A.',
8     database='decode labs_db'
9 )
10
11 cursor = conn.cursor()
12 cursor.execute('SELECT * FROM Interns;')
13
14 for row in cursor.fetchall():
15     print(row)
16
17 cursor.close()
18 conn.close()
```

Run query_interns

```
"C:\Program Files\Python311\python.exe" "D:\CS24006_08_11_15 (1)\query_interns.py"
(1, 'John Doe', 'Cloud Intern', 'jdoe@decode labs.com')
(2, 'Jane Smith', 'DevOps Intern', 'jsmith@decode labs.com')
(3, 'Conan O'Leary', 'Backend Intern', 'coleary@decode labs.com')

Process finished with exit code 0
```

11. Conclusion

This project demonstrated the responsibilities of a Cloud Infrastructure Engineer in building persistent, structured, and secure data storage: provisioning a managed RDS database, isolating it from public internet access, enforcing schema-level data integrity constraints, and safely accessing it through an

SSH-tunneled bastion host rather than exposing the database directly, reflecting real-world production security practices.